

Narratives from The Early History of F#

Don Syme

Principal Researcher and Community Contributor, Microsoft

@dsymetweets, fsharp.org/history

A meta-talk about a meta-language

Loosely based on HOPL IV paper

From 1970s to 2021

From Milner, Newey, Morris to Today

History is always written
selectively

I'll be explicit about the narratives
I've chosen

Narrative #1

A personal journey

Narrative #2

A community's journey
(and origin mythology)

Narrative #3

Even in PL things don't just happen. Things happen to people, who act, react and change in very human ways

creation, rejection, denial, stubbornness, cooperation, subversion, opportunism, ...

Narrative #4

What happens when computing traditions with strong belief systems collide, both in industry and academia?

Narrative #5

The dynamic interplay between academic research and industry concerns in computing history

Narrative #6

The contradictions and dialectics present throughout the origins and history of ML-family languages

Narrative #7

The golden thread –
what holds true in
programming
language design,
what stands the test
of time?



How ML Evolved

Robin Milner

1982

There are
contradictions at
the heart of the ML
family of languages

Purity v. Practicality
Logic v. Pragmatism

ML is one among many functional programming languages. But not many were designed, as ML was, for a more-or-less specific task. The point of this note is to summarise the process by which we were guided to ML, as it now is, by the demands of the task. We (at least I) feel that to find a good metalanguage for machine assisted proof, which was the task, we could hardly have gone in an essentially different direction; the task seemed to determine the language—and even made it turn out to be a general purpose language.

The context in 1974 (when the Edinburgh LCF project began) was our experience with the Stanford LCF proof assistant, developed there in 1971–72 by Richard Weyrauch, Malcolm Newey and myself. The development of ML as a metalanguage for interactive proof was the work of several people; in chronological order they were (besides me) Malcolm Newey, Lockwood Morris, Michael Gordon, and Christopher Wadsworth. Other people, who

ML does not use lazy evaluation; it calls by value. This was decided for no other reason than our inability to see the consequences of lazy evaluation for debugging (remember that we wanted a language which we could use rather than research into), and the interaction

The World as It Was (~1997)

ACM SIGPLAN *Functional Programming*

Why no one uses functional languages¹

Editor: Philip Wadler, Bell Laboratories, Lucent Technologies; wadler@research.bell-labs.com

Philip Wadler

To say that no one uses functional languages is an exaggeration. Phone calls in the European Parliament are routed by programs written in Ericsson's functional language Erlang. Virtual CDs are distributed on Cornell's network via the Ensemble system written in INRIA's CAML, and real CDs are shipped by Polygram in Europe using Software AG's Natural Expert. Functional languages are the language of choice for writing theorem provers, including the HOL system which helped debug the design of the HP 9000 line of multiprocessors. These applications and others are described in a previous column [1].

Still ... I work at Bell Labs, where C and C++ were invented. Compared to users of C, "no one" is a tolerably accurate count of the users of functional languages.

Advocates of functional languages claim they produce an order of magnitude improvement in productivity. Experiments don't always verify that figure — sometimes they show an improvement of only a factor of four. Still, code that's four times as short, four times as quick to write, or four times easier to maintain is not to be sniffed at. So why aren't functional languages more widely used?

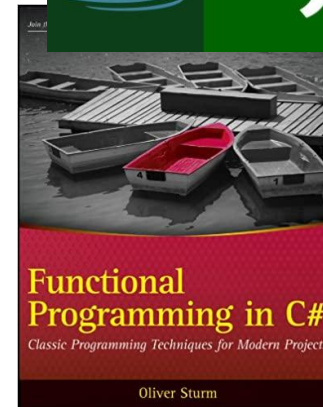
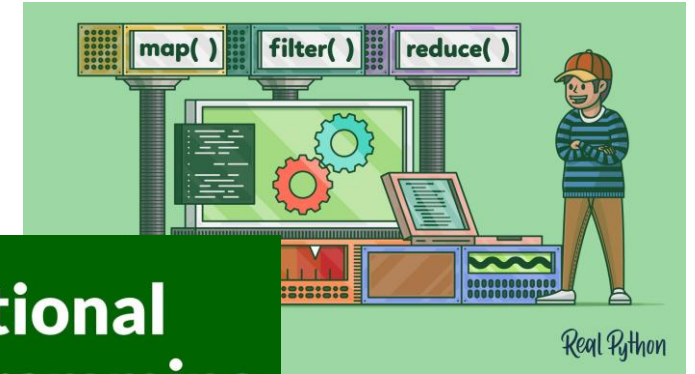
rather than built from scratch. Many of these components are written in C or C++, so a foreign function interface to C is essential, and interfaces to other languages can be useful.

The isolationist nature of functional languages is beginning to give way to a spirit of open interchange. Serious implementations now routinely provide interfaces to C, and sometimes other languages. Interworking with the imperative world is straightforward for strict languages like ML or Erlang, but trickier for lazy languages like Haskell or Clean, since laziness makes the order of evaluation difficult to predict. However, through a pleasing interplay of theory and practice, recent research has shown how abstract concepts such as monads or linear logic can be applied to smoothly interface lazy functional languages to the real world [2, 3].

Conquering isolationism is a task for everyone, not just functional programmers. The computing industry is now beginning to deploy standards, such as CORBA and COM, that support the construction of software from reusable components. Recent work allows any Haskell program to be packaged as a COM component, and any

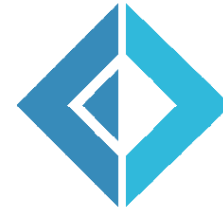
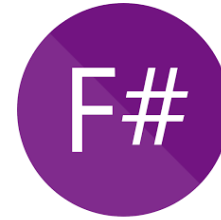
The World Today

Typed Functional Programming Techniques are Used Everywhere



The World Today

Strongly Typed Functional
Programming Languages
are Usable Everywhere



TypeScript



Rollback to 80s-90s

Strongly typed FP was small but active tribe.

Rooted in theorem proving, theory, Standard ML, Haskell, OCaml, experiments

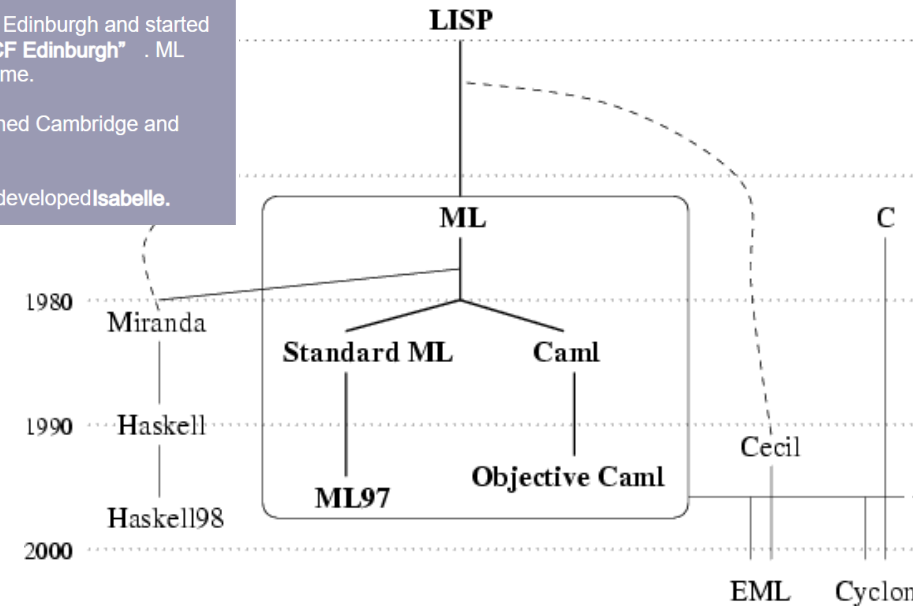


Milner

- British computer scientist.
- 1972: Milner developed proof checker for Scott's "Logic for Computable Functions" (LCF) at Stanford (known as "Stanford LCF").
- 1973: Milner moved to Edinburgh and started the successor project "LCF Edinburgh". ML language is born in this time.
- 1981: Mike Gordon joined Cambridge and HOL was born.
- 1990s: Larry Paulson developed Isabelle.



Paulson



Coq (software)

Poly/ML

The Poly/ML implementation of Standard ML.

Features

Full multiprocessor support in the thread library and garbage collector
Interactive debugger
Fast compiler
Preferred implementation for large projects including [Isabelle](#) and [HOL4](#).



Glasgow, Haskell and the **GHC** compiler

60 YEARS OF COMPUTING AT GLASGOW

The Haskell Programming Language

Haskell is a non-strict functional programming language widely used for research, and now increasingly adopted by industry. It is named after the logician Haskell Curry.

In 1987 the functional programming community formed a committee to define a standard non-strict language. Glasgow was well represented in the ~25-person committee by Kevin Hammond, John Hughes, Simon Peyton Jones, John Launchbury, and Philip Wadler.

The committee sometimes met in Glasgow and the first version of Haskell (1.0) was defined in 1990, and there have been a succession of standards since.



Philip Wadler



John Hughes

The Object-Oriented Tidal Wave, 1986-1998

	Oct	The C++ Programming Language [Stroustrup,1986]
1986	Aug	The "whatis paper" [Stroustrup,1986b]
	Sep	1st OOPSLA conference (start of OO hype centered on Smalltalk)
	Nov	1st commercial Cfront PC port (Cfront 1.1, Glockenspiel)
1987	Feb	Cfront 1.2

A story in itself, colossal impact

On Platform Companies (Obj-C, C++, Java)

On Microsoft (C++, Java)

On CS Academia (Java)

On Every Programming Language



Response 1

Find something
else to believe in

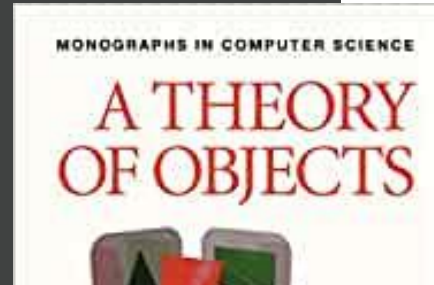
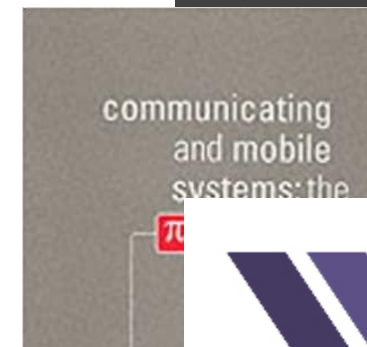


Type-theoretic foundations
for concurrent object-
oriented programming

Response 2

Retreat to the fort!
Focus on Foundations!
Focus on Pure
Functional! (and
variations)

I  VE
LOGIC



Foundations of Object
Workshop

Andrew Black[†]



Response 3

Verify the stuff using
functional/theory
tools!

Extended Static Checking for Java

Cormac Flanagan

Joint work
Mark Lillibridge
Jim Saxena
Compaq Systems

+ many other examples

The Static Driver Verifier Research Platform

Thomas Ball¹, Ella Bounimova¹, Vladimir Levin²,
Rahul Kumar², and Jakob Lichtenberg²

¹Microsoft Research

²Microsoft Windows

Spec#

Terminator

Automatically proving program termination

Byron Cook · MSR-Cambridge

<http://research.microsoft.com/Terminator>

Response 4

Oh yeah! Get my
language running on
those object VMs!



The MLj Compiler + many other examples

MLj is a complete system for SML to Java bytecode compilation. Its features include:

- conformance to a subset of [SML '97](#), approximately everything except for functors;
- implementation of a large subset of the [SML](#)
- typed-checked interlanguage working extension
- automatic recompilation management;
- whole-program optimisation to produce compact code



sml.net

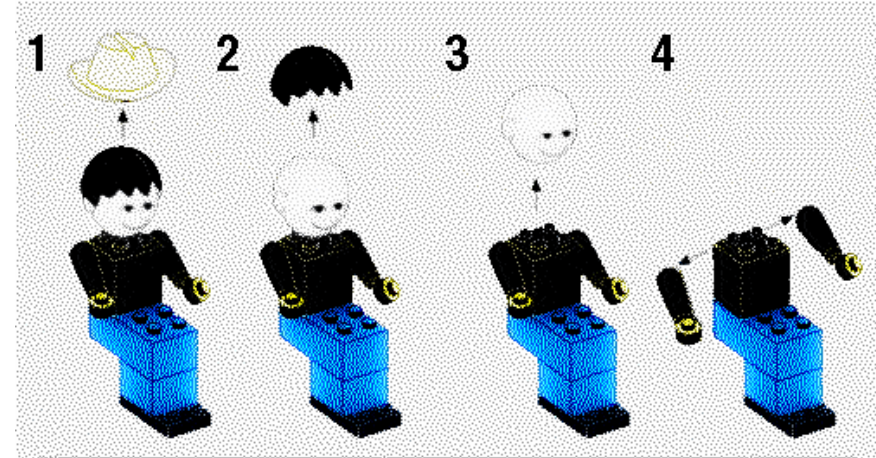
I've heard that there was a project where Microsoft

.NET and then it was replaced with the F# project. Is that true? If it's true, why?

That's a small part of the sequence. The visional design of the .NET platform was very much expected to be a multilanguage platform from the start. Right back in 1998, just in fact as our research group in programming languages started at

Response 5

Rationally deconstruct Functional and Object Programming



20+ features of OO

1. dot notation (`x.Length`)
2. instance members
3. type-directed name resolution
4. implicit constructors
5. static members
6. indexer notation `arr[x]`
7. named arguments
8. optional arguments
9. interface types
10. mutable data
11. defining events
12. defining operators on types
13. auto properties
14. `IDisposable`, `IEnumerable`
15. type extensions
16. structs
17. delegates
18. enums
19. implementation inheritance
20. `nulls` and `Unchecked.defaultof<_>`
21. method overloading
22. curried method overloads
23. protected members
24. `self` types
25. wildcard types
26. aspect oriented programming ...
27.

???

Response 6a

Seek a synthesis!

Make Functional
Languages more
Object-Oriented!



Dylan

O'Haskell

O'Haskell is Haskell conservatively extended with static subtyping and monadic object

- [O'Haskell](#), as archived by the Wayback Machine



Response 6b

Seek a synthesis!

Make Object-
Oriented
Languages more
Functional!

A slice of Pizza: A quick introduction to Pizza, a dialect of Java

Martin Odersky
University of Southern Australia - Philip Wadler
Lucent Technologies

10 October 1997

Pizza is a superset of Java that incorporates four additional features.

- *Parametric polymorphism* : the ability to parameterize classes
- *Higher-order functions* : the ability to treat functions as values
- *Algebraic data types* : a convenient way of representing trees
- *Tail calls* : don't grow the stack when the last operation in one

Cw

Scala



+ many other more recent examples



Meanwhile.... Over at Microsoft....



Microsoft®
Research



Microsoft®
.net

Microsoft goes to Cambridge

Roger Needham has a dream: research, not to do anything for them. But as head of Microsoft's new research laboratory he already has plans to make computers self-managing, secure and share all power as well. **John Kananagh reports**

When Microsoft's chief research officer, Roger Needham, arrived in Cambridge, he found the city's computer scientists were already doing what he wanted to do. He found that they were not only doing it, but they were doing it better than he was.

It's not that they were not doing it, but they were doing it better than he was. He found that they were not only doing it, but they were doing it better than he was.

He found that they were not only doing it, but they were doing it better than he was. He found that they were not only doing it, but they were doing it better than he was.

He found that they were not only doing it, but they were doing it better than he was. He found that they were not only doing it, but they were doing it better than he was.

He found that they were not only doing it, but they were doing it better than he was. He found that they were not only doing it, but they were doing it better than he was.

He found that they were not only doing it, but they were doing it better than he was. He found that they were not only doing it, but they were doing it better than he was.

He found that they were not only doing it, but they were doing it better than he was. He found that they were not only doing it, but they were doing it better than he was.

He found that they were not only doing it, but they were doing it better than he was. He found that they were not only doing it, but they were doing it better than he was.

Leading Edge: Cambridge



Roger Needham is the head of Microsoft's new research laboratory in Cambridge. He is a leading expert in the field of self-managing computers.

Microsoft's new research laboratory in Cambridge is a leading expert in the field of self-managing computers. It is a leading expert in the field of self-managing computers.

Microsoft's new research laboratory in Cambridge is a leading expert in the field of self-managing computers. It is a leading expert in the field of self-managing computers.

Microsoft's new research laboratory in Cambridge is a leading expert in the field of self-managing computers. It is a leading expert in the field of self-managing computers.

Microsoft's new research laboratory in Cambridge is a leading expert in the field of self-managing computers. It is a leading expert in the field of self-managing computers.

Microsoft's new research laboratory in Cambridge is a leading expert in the field of self-managing computers. It is a leading expert in the field of self-managing computers.

Microsoft's new research laboratory in Cambridge is a leading expert in the field of self-managing computers. It is a leading expert in the field of self-managing computers.

Microsoft's new research laboratory in Cambridge is a leading expert in the field of self-managing computers. It is a leading expert in the field of self-managing computers.

1997 – Microsoft Research
founded in Cambridge

All the tribes of computer science
descend on Microsoft..

The worlds of Robin Milner and Bill
Gates collide...

Much early work centred on .NET

.NET Generics (1998-2004)

Breaking through the
nominal, class-based
Object-Oriented wall

Influenced by SML, OCaml, more + GJ, Pizza by
Wadler/Odersky et al

“Puts .NET 20 years ahead”

MSR White Paper: Proposed Extensions to COM+ VOS (Draft)

*Don Syme, Nick Benton, Simon Peyton-
Jones, Cedric Fournet*

1.1	Getting Serious about Language Innovation	
1.1.1	Introduction	5
1.1.2	Compound Types	6
1.1.3	Function Types, Closures and Thunks (= Generalized Delegates)	6
1.1.4	Parametric Polymorphism	6
1.3	Halfway is Not Good, but may have to do	7
2	Compound Types	8
2.1	Very Simple and Very Useful	8
2.2	An Example	9
2.3	Compound Types are needed anyway	10
2.4	Implementation Path A – Full Support	11
2.5	Implementation Path B – Support by Agreement	11
3	Parametric Polymorphism	12
3.1	Introduction	12
3.1.1	PP, Source Languages and the CLS	12
3.1.2	Overview	13
3.2	Parametric Types	13
3.3	Examples of Implementing Parametric Types	14
3.3.1	Collection Classes	14
3.3.2	Polymorphic Methods	18
3.3.3	Comparison and Sorting	19
3.3.4	Printable Lists via Bounded Parameters	20
3.3.5	Cloneable Lists	20
3.3.6	Closures and Map	21
3.4	Further Details	22
3.4.1	More on Instantiations	22

By 2003 .NET Generics was safely landed

However, C# 2.0 was still verbose, unusable,
untrustworthy (relative to FP)

Sample after sample showed 3x-5x code size
difference for like-for-like, full of null checking etc.

A later example

350,000

lines of C# OO
by offshore team

The C# project took five years and peaked at ~8 devs. It never fully implemented all of the contracts.

The F# project took less than a year and peaked at three devs (only one had prior experience with F#). All of the contracts were fully implemented.

30,000

lines of robust F#, with
parallel + more features

An application to evaluate the revenue due from [Balancing Services](#) contracts in the UK energy industry

<http://simontcousins.azurewebsites.net/does-the-language-you-use-make-a-difference-revisited/>

Implementation	C#	F#
Braces	56,929	643
Blanks	29,080	3,630
Null Checks	3,011	15
Comments	53,270	487
Useful Code	163,276	16,667
App Code	305,566	21,442
Test Code	42,864	9,359
Total Code	348,430	30,801

I loved OCaml, by far the best practical language at that time

F# 1.0 was born from an obsession to ensure practical strongly-typed FP was a viable option in the 21st C

Starting with Core Caml

Orthogonal & Unified Constructs

→ Let “let” simplify your life...

Type inference. The safety of C# with the succinctness of a scripting language

Bind a static value

Bind a static function

Bind a local value

Bind an local function

```
let data = (1,2,3)

let f a b c =
  let sum = a + b + c
  let g x = sum + x*x
  g a , g b , g c
```



OCaml



.NET Runtime

(This is a truck engine)

The F# approach

Start with Core Caml

Stay true to core FP principles

Interoperate

Reuse industrial VM+ecosystem

Deconstruct OO, FP, find synthesis

Include IDE tooling, REPL

Deliver and apply in industry

Long-term home at Microsoft (Research)

Focused iterative improvements

Parameterization

Type definitions,
modelling,
objects

Soundness and
Formalization

Expression-oriented
programming

Runtime,
ecosystem,
interop

Computational
modalities
(comprehensions, async,
monads, monoids...)

Types and Type
Inference

Nominal objects (classes, interfaces)

Structural objects

Languages must have a formal specification!

Functors
(parameterized large
components)

Type definitions

Functional objects

Soundness supreme

Functional Parameterization abstraction and parameterization

Immutable by default, algebraic data

Initialization
soundness, no nulls

Expression-oriented programming

“F# types are .NET types, .NET types are F# types”

Types for modelling, perf, correctness, interop

Hindley-Milner for typed succinctness

Types and Type Inference

Fast generic unboxed arith + inline

Type-classes and type-level computation

Types for logic and proof

List comprehensions

Monadic notation

Computational

**Rich, compositional computation
notation** (for comprehensions,
queries, async, monads, monoids,
applicatives...)

Interop

Type Providers

Facing up to some of the Contradictions of
the Strongly Typed Functional Tribe

How ML Evolved

Robin Milner

1982

There are
contradictions at
the heart of the ML
family of languages

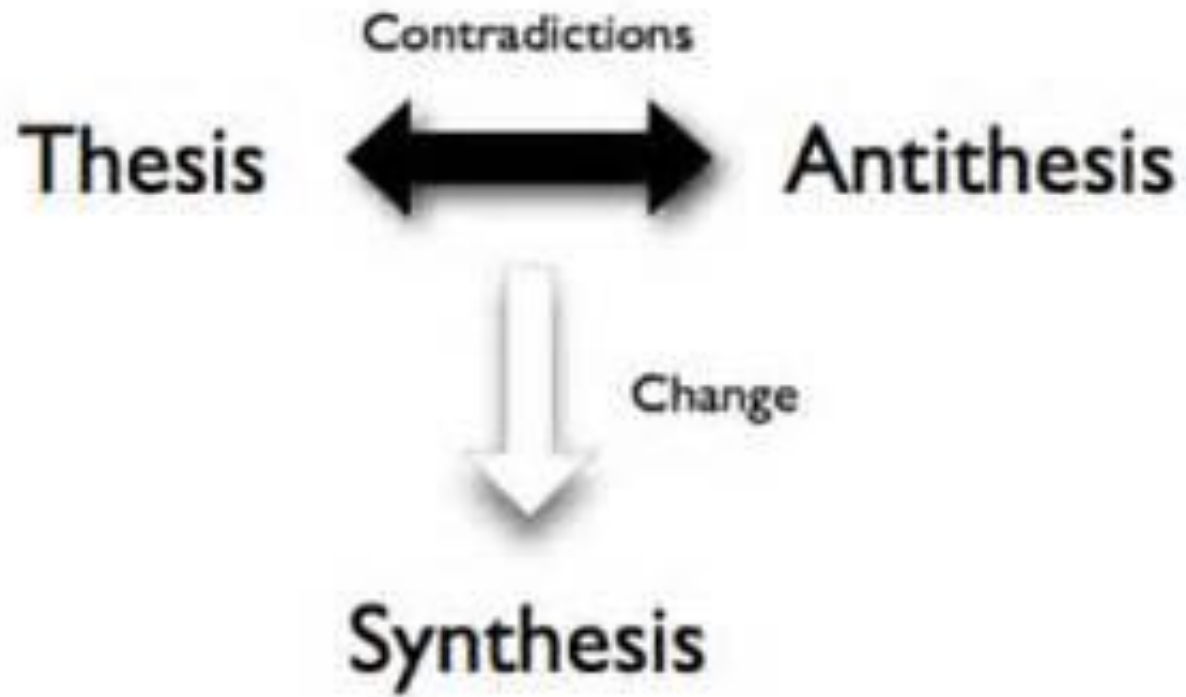
Purity v. Practicality
Logic v. Pragmatism

ML is one among many functional programming languages. But not many were designed, as ML was, for a more-or-less specific task. The point of this note is to summarise the process by which we were guided to ML, as it now is, by the demands of the task. We (at least I) feel that to find a good metalanguage for machine assisted proof, which was the task, we could hardly have gone in an essentially different direction; the task seemed to determine the language—and even made it turn out to be a general purpose language.

The context in 1974 (when the Edinburgh LCF project began) was our experience with the Stanford LCF proof assistant, developed there in 1971–72 by Richard Weyrauch, Malcolm Newey and myself. The development of ML as a metalanguage for interactive proof was the work of several people; in chronological order they were (besides me) Malcolm Newey, Lockwood Morris, Michael Gordon, and Christopher Wadsworth. Other people, who

ML does not use lazy evaluation; it calls by value. This was decided for no other reason than our inability to see the consequences of lazy evaluation for debugging (remember that we wanted a language which we could use rather than research into), and the interaction

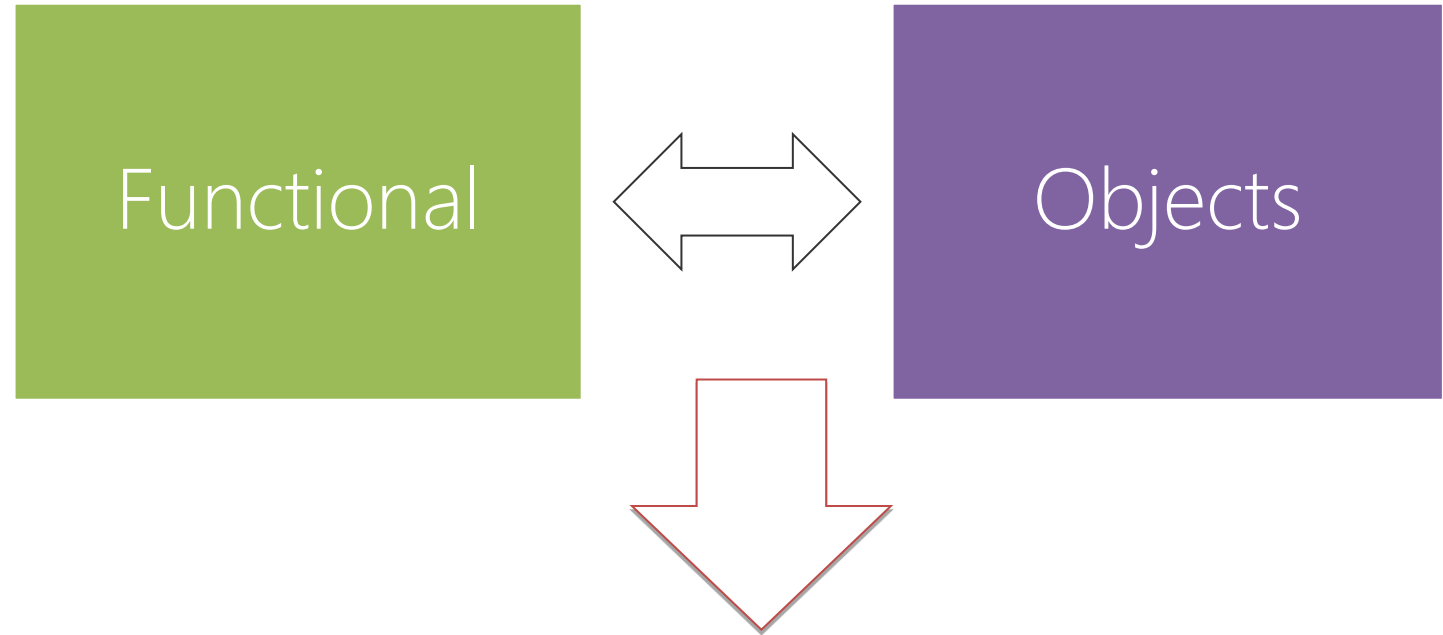
Embracing Dialectics



Contradiction #1

All practical typed languages
need cheap and effective type
declarations that include
encapsulation

Previous MLs had awkward
variants of that (abstype)



2006: F# Object Programming

modified Hindley-Milner, type-directed name resolution,
nominal, subtype-friendly, delegation-oriented, expression-
friendly

F# : Objects + Functional

```
type Vector2D(dx:double, dy:double)
```

Recovers ergonomic simplicity
for encapsulation, works with type inference
and interop, immutable by default

```
let d2 = dx*dx+dy*dy
```

**Inputs to object
construction**

```
member _.DX = dx
```

Functional object internals

```
member _.DY = dy
```

Exported properties

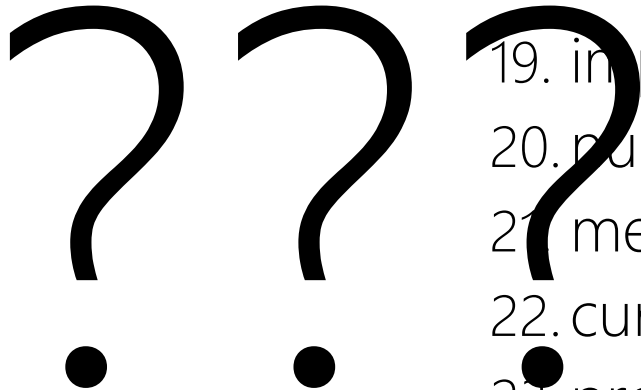
```
member _.Length = sqrt d2
```

Exported method

```
member _.Scale(k) = Vector2D (dx*k, dy*k)
```

Deconstructing Object Programming

The 20+ features of OO

1. dot notation (`x.Length`)
 2. instance members
 3. type-directed name resolution
 4. implicit constructors
 5. static members
 6. indexer notation `arr.[x]`
 7. named arguments
 8. optional arguments
 9. interface types
 10. mutable data
 11. defining events
 12. defining operators on types
 13. auto properties
 14. `IDisposable`, `IEnumerable`
 15. type extensions
 16. structs
 17. delegates
 18. enums
 19. implementation inheritance
 20. `nulls` and `Unchecked.defaultof<_>`
 21. method overloading
 22. curried method overloads
 23. protected members
 24. `self` types
 25. wildcard types
 26. aspect oriented programming ...
 27. ...
- 

Some make F# a better API language

Some make F# a better implementation language

Some are part of an interop standard

Some are not needed

The 20+ features of OO

Embrace

1. dot notation (`x.Length`)
2. instance members
3. type-directed name resolution
4. implicit constructors
5. static members
6. indexer notation `arr[x]`
7. named arguments
8. optional arguments
9. interface types and impl
10. mutable data
11. operators on types
12. auto properties
13. `IDisposable`, `IEnumerable`
14. type extensions
15. events

Use where
necessary, use
tastefully, use
respectfully, use
sparingly

16. structs
17. delegates
18. enums
19. type casting
20. large type hierarchies
21. implementation inheritance
22. nulls and `Unchecked.defaultof<_>`
23. method overloading
24. curried method overloads
25. protected members
26. self types
27. wildcard types
28. aspect oriented programming ...
29. ...

Down the object
rabbit hole

Not supported

The 20+ features of OO

1. dot notation (`x.Length`)
2. instance members
3. type-directed name resolution
4. implicit constructors
5. static members
6. indexer notation `arr[x]`
7. named arguments
8. optional arguments
9. interface types and implementations
10. mutable data
11. operators on types
12. auto properties
13. `IDisposable`, `IEnumerable`
14. type extensions
15. events

Love

Tolerate

16. structs
17. delegates
18. enums
19. type casting
20. large type hierarchies
21. implementation inheritance
22. nulls and `Unchecked.defaultof<_>`
23. method overloading
24. curried method overloads
25. protected members
26. self types
27. wildcard types
28. aspect oriented programming ...
29. ...

Mostly Avoid

Forget

Object Programming v. Object-Oriented Programming

Object Programming focuses on ...

succinct coding

notational convenience

API ergonomics

good naming

practical encapsulation

sensible, small, composable abstractions

expression-oriented object implementations

delegation and closure

making simple things out of (potentially complex) foundations

Object-Oriented Programming seems to focus on ...

objects as a single, dominant, mentally stifling paradigm

pointless hierarchical classification (Animal, Cat, Dog, AbstractJellyBeanFactoryDelegator)

large, cumbersome abstractions with many holes and failure points

declarations not expressions

composition through... more objects, more hierarchies, more names

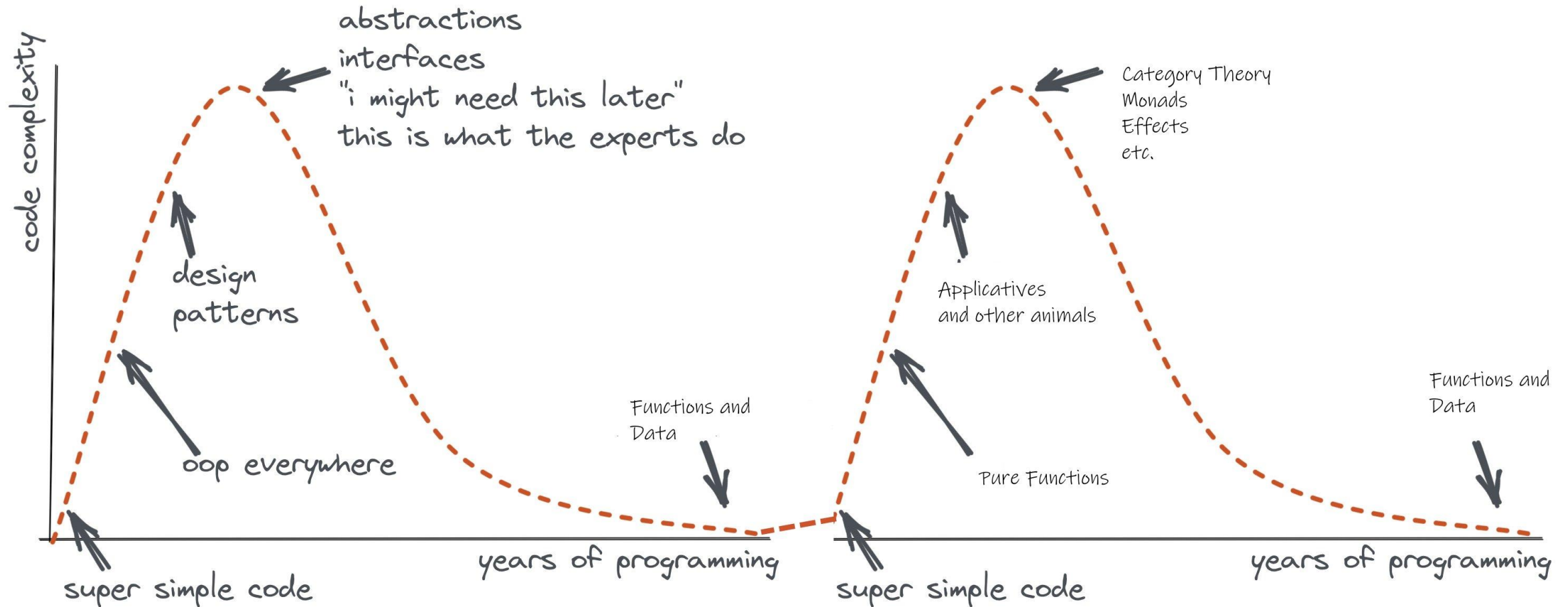
making complex things out of complex foundations

The F# approach is to embrace object programming, make it fit with the expression-oriented typed functional paradigm

but not embrace full “object-orientation” (unless you happen to be in a project using that technique)

Remember the eternal sweet spot of programming!

Functions + Data + Types

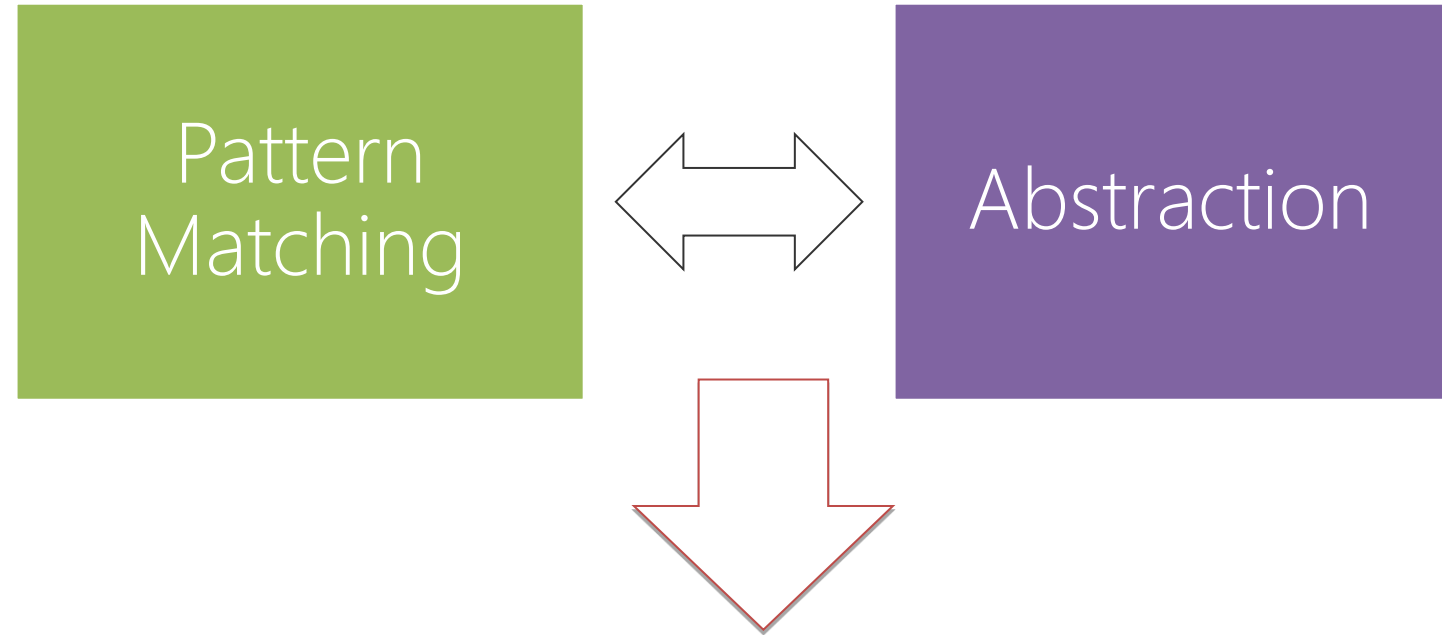


(3) ML does not adopt the clausal form of function definition, which is found so convenient by users of HOPE and PROLOG. How can we get a semantically rigorous form of this clausal definition, in which the constructor-patterns in formal parameters can involve not only primitive constructors, but also the constructors of user-defined abstract types? The problem is to know that these constructors are constructors, in the sense of being uniquely decomposable (or else to admit non-https://www.pure.ed.ac.uk/ws/portalfiles/portal/17084823/Milner_R_1982_How_ML_Evolved.pdf

Contradiction #2

Pattern matching only over concrete representations is, in practice, a catastrophe

This was a contradiction in all functional languages until F#



2006: F# Active Patterns

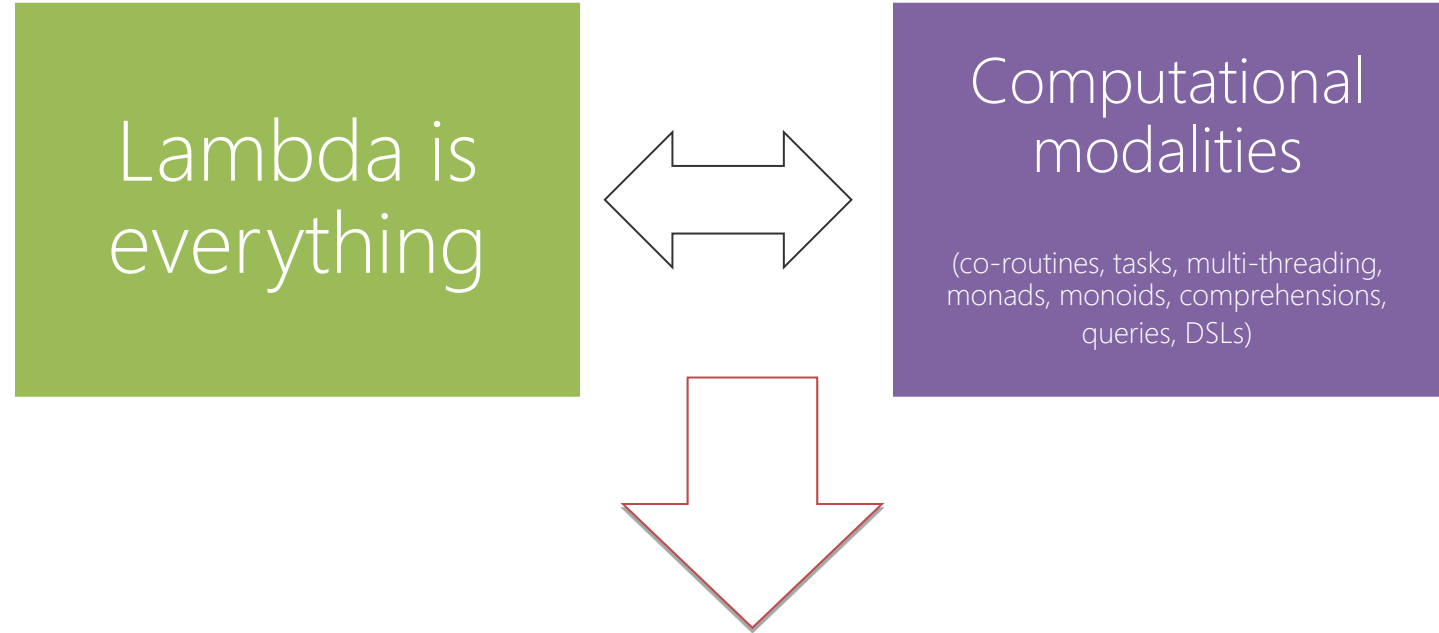
Views made practical and simple.
Concomitant with Scala extractors, leads to Haskell view patterns (less used), C# pattern matching

Contradiction #3

MLs had no list
comprehensions

MLs had no monad syntax

MLs had nothing like this, and
it really hurt in practice



1993: OCaml embraces Algol-like control syntax within an
expression-based functional language

2007: F# Comprehensions, Async and Computation
Expressions gives modal interpretation to this

Computational modalities in C#, F# (computation expressions)

C# Iterator methods (from Icon)

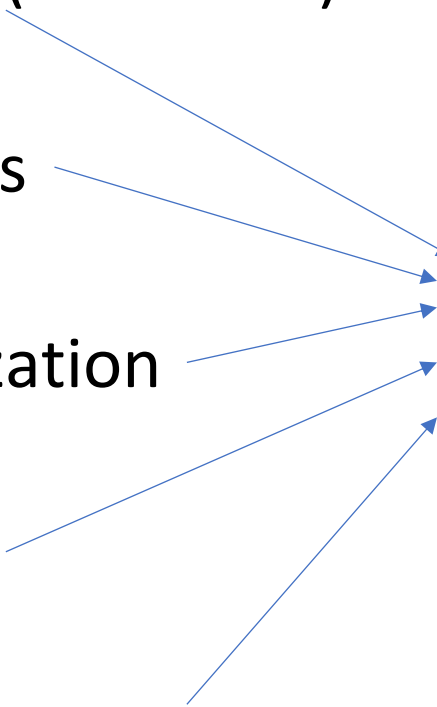
C# Query expressions

C# Collection initialization

C# Async methods

C# Async iterator methods

modality { code }



```
graph LR; A[C# Iterator methods (from Icon)] --> E[modality { code }]; B[C# Query expressions] --> E; C[C# Collection initialization] --> E; D[C# Async methods] --> E; F[C# Async iterator methods] --> E;
```

2007 – monads, monoids, async, comprehensions

2012 – queries, extended comprehensions

2020 – applicatives

Computational modalities in C#, F# (computation expressions)

C# Iterator methods

`seq { code + yield }`

C# Query expressions

`query { code + yield + custom }`

C# Collection initialization

`list { code + yield }`

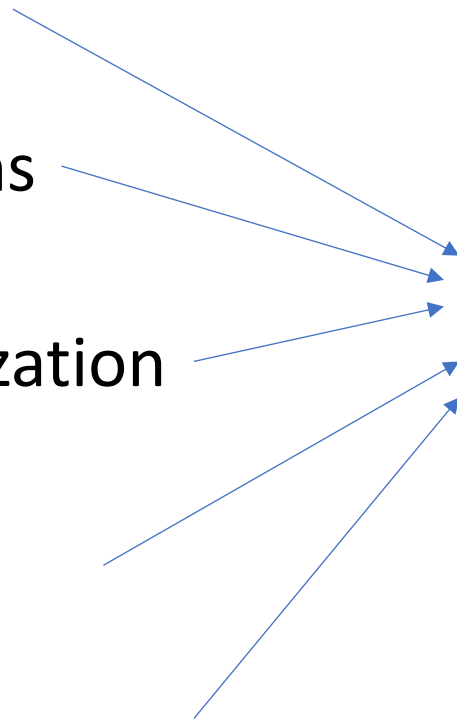
C# Async methods

`async { code + let! }`

C# Async iterator methods

`asyncSeq { code + let! + yield }`

`coroutine { code + yield() }`



seq { ... }, [...], [| ... |]



```
let rec allSymbolsInEntities compGen (entities: FSharpEntitylist) =
    [ yield! entities

      for gp in e.GenericParameters do
        if compGen || not gp.IsCompilerGenerated then
          yield gp

      for x in e.MembersFunctionsAndValues do
        if compGen || not x.IsCompilerGenerated then
          yield x

        for gp in x.GenericParameters do
          if compGen || not gp.IsCompilerGenerated then
            yield gp

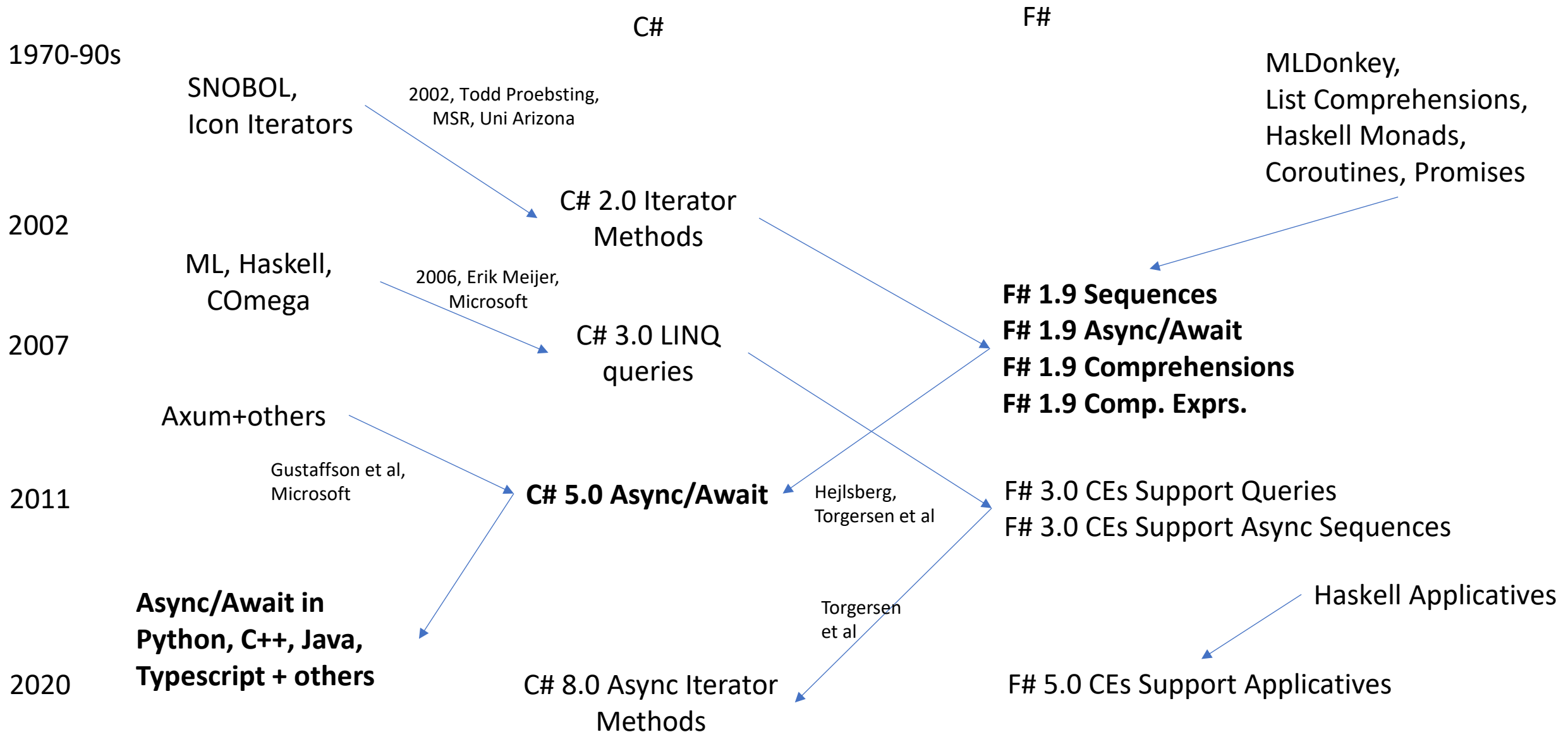
      yield! e.UnionCases

      for f in x.UnionCaseFields do
        if compGen || not f.IsCompilerGenerated then
          yield f

      for x in e.FSharpFields do
        if compGen || not x.IsCompilerGenerated then
          yield x

      yield! allSymbolsInEntities compGen e.NestedEntities ]
```

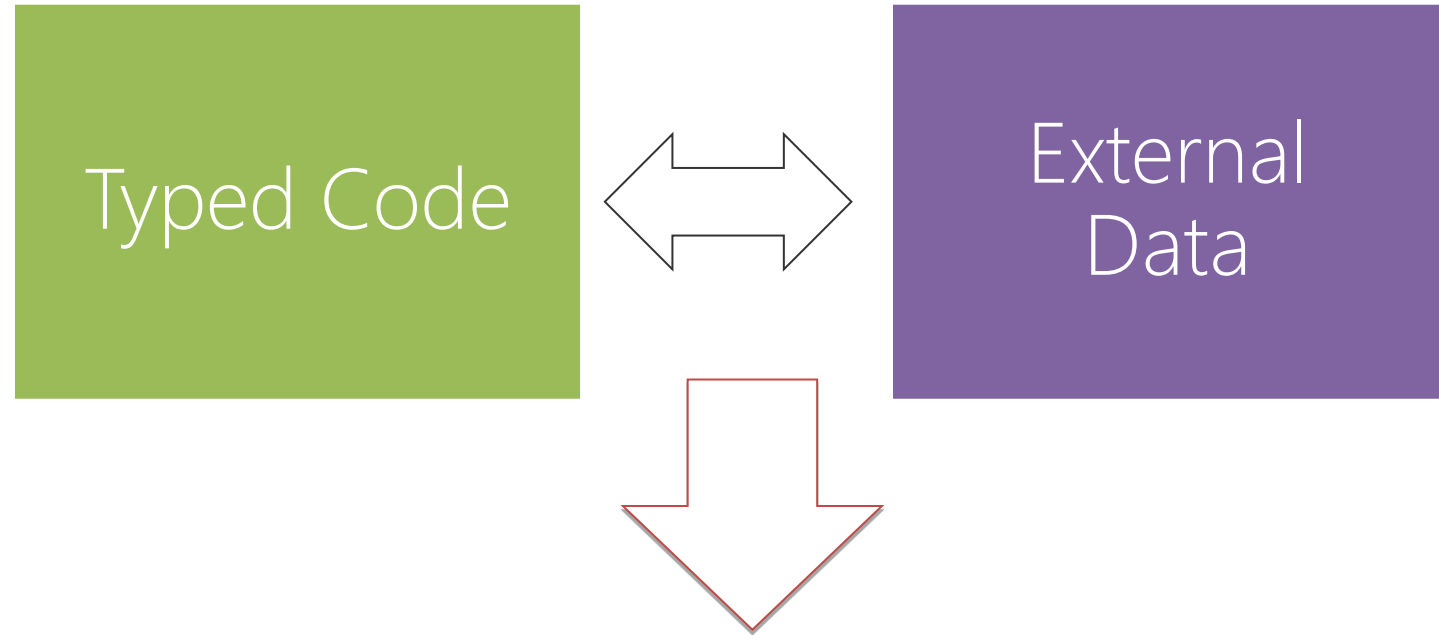
History of async/await and computational modalities in C#, F#



Contradiction #4

No versions of ML had taken data-access or data interop seriously

F# had some libraries from .NET. However we wanted more fluency, more automation, more programmability of interop



2012: F# Type Providers
(compile-time meta-programming to compute type spaces based on external information sources)

The More Recent History of F# (2012+)

- So much I could say
- 2010 – F# 2.0, “Parachutes Into the Dark Heart of the Industry”
- 2012+ – F# 3.0, 3.1
- 2014-2016 – Fully embrace Cross-Platform, Open-Source, Cloud, Web
- 2016+ – F# 4.0, 4.1, 4.5, 4.7
- 2020 – F# 5.0
- 2021 – F# 6.0 in preparation

F# Today

- F# has a core position as “typed functional for .NET and JavaScript”
 - Delivery in the .NET SDK, F# exists wherever C# exists
 - Highly performant
 - IDE tooling by two major vendors
- Strong for web-client, mobile, server, cloud, deep-learning
- Fully open design and engineering processes
- A practical functional-first methodology
- An engaged and enjoyable community

F# get started

- `dotnet new -lang F#`

`dotnet build`

F# tools are part of
the .NET SDK,
available everywhere



www.dot.net

F# for the web backend

- `dotnet new -i "giraffe-template::*"`
`dotnet giraffe`

High perf, functional
server-side
programming



A functional ASP.NET Core micro
web framework for building rich
web applications.

github.com/giraffe-fsharp/Giraffe

F# for the web frontend (JavaScript)

- `dotnet new -i "Fable.Template::*"`

```
dotnet new fable  
npm install  
npm start
```

You can use F# as a
JavaScript language



fable.io



websharper.com

F# for the full stack with SAFE

- `dotnet new -i SAFE.Template`

```
dotnet new SAFE  
dotnet tool restore  
dotnet run
```



Lambdas in the roof?



The influence of F#

Culturally, F# core team existed “right next door” to C# core team, leading to many direct influences both ways

via Hejlsberg, Meijer, Hoban, Parsons, Torgersen + others

C# 3.0	- Auto-implemented properties - Anonymous types - Query expressions - Lambda expressions - Expression trees - Extension methods - Implicitly typed local variables - Partial methods - Object and collection initializers
C# 4.0	- Dynamic binding - Named/optional arguments - Generic covariant and contravariant - Embedded Interop types
C# 5.0	- Asynchronous members - Caller info attributes
C# 6.0	- Static imports - Exception filters - Auto-property initializers - Expression bodied members - Null propagator - String interpolation - nameof operator - Index initializers - Await in catch/finally blocks - Default values for getter-only properties
C# 7.0	- Out variables - Tuples and deconstruction - Pattern matching - Local functions - Expanded expression bodied members - Ref locals and returns - Discards - Binary literals and digit separators - Throw expressions

- C# 7.1
- async Main method
 - default literal expressions
 - Inferred tuple element names
 - Pattern matching on generic type parameters

- C# 7.2
- Techniques for writing safe efficient code
 - Non-trailing named arguments
 - Leading underscores in numeric literals

The \$1B mistake is finally corrected.

Real cost \$100B??

.NET libraries now come with ubiquitous nullability annotations

- Extension methods
- String enhancements
- Annotations
- Patterns
- Tuple
- Positional
- Patterns

- Using declarations
- Static local functions
- Disposable constructs
- Nullable reference types
- Asynchronous streams
- Indices and ranges
- Null-coalescing assignment
- Unmanaged constructed types
- Stackalloc in nested expressions
- Enhancement of interpolated verbatim strings

+ C# 9.0 records, top-level statements

Other Influences of F#...

Scala (via Odersky + others – extractors + more)

TypeScript, Swift, Elm, Kotlin (F# is a reference point)

Python (via C# - async)

Java, C++ ... (via C# - async)

Concluding Remarks

Concluding Notes

- The programming world has corrected the OO dominance of the 90s
 - **Generics, lambdas, parameterization, immutability, algebraic types, various computational modalities, type inference** have now rightly occupied their place as practical core techniques in nearly all programming languages
- “Why no one uses Functional Programming” has been consigned to history
- F# has been a part of this story since 2003
 - Part of the reaction of the functional tribe to the OO tidal wave
 - Stayed true to its core design goals
 - Many practical additions to the expression-oriented, statically typed paradigm
 - A strong user base and a secure future

The golden thread of ML is the simplicity of Functions + Data, e.g.

```
let f x = (x, x)
type term = T of string * term list
...
```

Milner, Newey, Morris saw this in 1974 and it's stayed true.



Thank you!!

Questions?